ID_2 Toolkit Script Examples Companion

APOD-ID2-SCRIPT-EXAMPLES-002

Informative • Non-Normative • Optional Companion

Applies to: Access-Mode ID_2 sessions using optional JSON records and PowerShell 7
No coding required: Scripts are executed via copy/paste commands; no script modification is required.

Document Information

Document Identifier: APOD-ID2-SCRIPT-EXAMPLES-002
Title: Access-Mode ID_2 — Operational / Runtime Reasoning: Toolkit Script Examples Companion
Status: Informative Companion (Non-Normative)
Date of Issue: 18 February 2026
Author: Anthony Siljeg
Affiliation: Alnotrea Labs — Policy of Developments (Access PoD Initiative)
Execution Environment: PowerShell 7
Access-Mode Posture: ID_2 — Operational / Runtime Reasoning (Exploratory, Non-Inspection)

Authority Note

This document supplements the six ID_2 modules.
It does not modify, replace, or elevate their posture.

Use of JSON records and scripts is optional and does not convert ID_2 into an inspection, audit, compliance, or certification mode.

All outputs remain exploratory and require human judgment.

Reader Orientation — How to Use This Companion

This companion extends **Access-Mode ID_2 — Operational / Runtime Reasoning** by providing optional structured recording and comparison tooling.

ID_2 itself is a governance reasoning discipline focused on stress-testing scope boundaries, authority drift, escalation logic, and uncertainty preservation under operational pressure.

The six Markdown modules remain the primary artifacts of ID_2.

This document introduces:

- Optional JSON session records

- Structured capture of stress simulation signals
- PowerShell 7 helper scripts
- Baseline → variant comparison workflows

These tools enhance repeatability and structured comparison.
They do not redefine or elevate the posture of ID_2.

When to Use This Toolkit

Use this companion when you want:

- Structured capture of reasoning signals across sessions
- Reproducible stress-test runs
- Structured baseline vs variant comparisons
- JSON export artifacts for workshop or internal review

You do not need this toolkit to run ID_2.
The modules can be completed fully in Markdown form without JSON or scripts.

What This Toolkit Is Not

This document is:

- Informative
- Non-normative
- Exploratory
- ID_2 posture only

It does not:

- Produce compliance determinations
- Assert adequacy or sufficiency
- Freeze inspection posture
- Replace human judgment
- Convert signals into decisions

All interpretation and conclusions remain human.

Structure of This Companion

The workflow follows a structured progression:

1. Scaffold a session container
2. Populate baseline session.json
3. Populate Module 03 (stress simulation)
4. Optionally create a stress-intensified variant
5. Normalize JSON structure (if comparing)
6. Run exploratory comparative analysis

The comparison pipeline surfaces signal differences, not verdicts.

Important: Optional JSON Layer

The JSON files in `/records/` are illustrative and lightweight.
They exist to:

- Capture reasoning signals in structured form
- Prevent silent schema drift
- Enable structured comparison

They are optional.
ID_2 does not depend on JSON to function as a discipline.

Release Context

This release includes a fully populated exemplar session under:

```
records\sessions\2026-02-10\
```

This exemplar demonstrates canonical structure and repeatability.

Execution Expectation

A complete baseline → variant → comparison cycle typically requires 60–90 minutes.

Repeat runs are encouraged to strengthen interpretive discipline and boundary enforcement reflexes.

Contents

Session scaffolding from templates (why this step exists) 8
Test run (primer) identifiers are used throughout this document 8
Step A: Create the baseline scaffold (folders) in PS7 8
Step B: Copy templates into the baseline scaffold (robocopy) 9
Step C: Create a matching variant scaffold - time pressure. 10

STEP 1 — Populate the Baseline Session Record 15
1. Session ID 15
2. Timezone 16
3. Facilitator Name 16
4. Facilitator Role 16
5. Participant Name(s) 17
6. Project Name 17
7. Audience 18
8. Purpose Statement 18
9. In Scope 18
10. Out of Scope 19
11. Run Plan Notes 19
12. Artifacts List 20
13. Signed By (Optional) 20
What Happens When You Save 20
Baseline Session Is Now Established 20

STEP 2 — Populate Module 03 (Baseline Simulation) 21
SECTION A — Inputs 21
1. Scenario Seeds 21

2. Stressors 22
3. Context Notes 22
SECTION B — Captures 23
4. Objective 23
5. Scope In 23
6. Scope Out 23
7. Assumptions 24
8. Evidence to Capture 24
9. Controls & Guardrails 24
SECTION C — Simulation Run 25
10. Run ID 25
11. Scenario Summary 25
12. Stress Conditions — Example Sets 26
A. Time Pressure 26
B. Telemetry Ambiguity 26
C. Stakeholder Pressure 27
D. Definition Drift 27
E. Authority Boundary Stress 27
F. Communication Friction 28
How to Select Stress Conditions 28
Example Baseline Stress Conditions (Clean ID_2 Form) 28
13. Branch Points 29
14. Signals Observed 29
15. Fragility Points 30
16. Authority Drift Flags 30
17. Uncertainty Notes 31
18. Escalation Triggers 31
19. Stop or Reset Events 32
What Not to Do 32

Two Types of Events 32
No Stop / No Reset 32
Stop Event (Pause) 33
Reset Event (Scope Re-declared) 33
20. Ethical Constructs 34

STEP 3 — Create and Populate Variant (Time-Pressure Intensified) 35
Important ID_2 Reminder 35
3.1 Create the Variant Session Folder 36
3.2 Session Template Copy — Future Use Guidance 37
3.3 Populate the Variant Session Record – Test-run 38
Field-by-Field Guidance for Variant Session 38
3.4 Create the Variant Module 03 Record 42
3.5 — Populate Variant Module 03 (Intensified Stress) 44
SECTION A — Inputs (Variant) 45
3) Context Notes (Variant) 46
SECTION B — Captures (Variant) 46
4) Objective 46
5) Scope IN 46
6) Scope OUT 47
7) Assumptions 47
8) Evidence to Capture 47
9) Controls/Guardrails 47
SECTION C — Simulation Run (Variant) 47
10) Run ID 47
11) Scenario Summary (Variant) 48
12) Stress Conditions (Variant) 48
13) Branch Points 48
14) Signals Observed 49

15) Fragility Points 49
16) Authority Drift Flags 49
17) Uncertainty Notes 49
18) Escalation Triggers 50
19) Stop or Reset Events 50
20) Ethical Constructs (Variant) 50

STEP 4 — Comparison Pipeline 53
4.1 Create the Comparison Output Folder 53
4.2 Normalize Module 03 JSON Records 53
4.3 Run Comparative Analysis and Export Summary 54
4.4 Final Verification 54

Troubleshooting — Schema Warning: 56
Canonical JSON Structure — Required Invariants 58

STEP 5 — Summary Checklist (Completion Review) 60
1. Session Structure 60
2. Canonical JSON Integrity 60
3. Comparison Pipeline 60
4. Export Confirmation 61
5. Posture Confirmation (ID_2 Boundary) 61
Completion Standard 62

Session scaffolding from templates (why this step exists)

Before running any ID_2 scripts, you want a **stable, predictable session skeleton** on disk. This matters for three reasons:

1. **Path determinism:** the primary scripts assume a consistent session layout (date → baseline/variants → session ID → artifacts). If folders drift, script prompts become ambiguous and records fragment.
2. **Template fidelity:** starting from the shipped templates prevents "silent drift" (missing keys, wrong casing, wrong list/object types) that later produces schema warnings and noisy diffs.
3. **Traceable runs:** the session ID becomes your **unit of analysis**. Every subsequent edit, run, normalization, export, and comparison is anchored to that one ID.

This section creates a "test-run" primer session scaffold using **robocopy** so the doc workflow matches your current folder structure and your scripts have the exact files they expect.

Test run (primer) identifiers are used throughout this document

First run Current folder setup on page 63. Use these primer identifiers consistently:

- **Session date folder:** `2026-02-16`
- **Baseline session ID:** `ID2-20260216-0900-test-run`
- **Variant naming rule (if used):** add a suffix such as `-timepressure`

Target baseline artifacts folder:

```
$workingRoot\records\sessions\2026-02-16\baseline\ID2-20260216-
0900-test-run\artifacts
```

The commands below create this path in your working copy.

Step A: Create the baseline scaffold (folders) in PS7

Run from the ID_2-Access working copy prepared on page 63. All relative paths below resolve from

```
$workingRoot
```

Use this working copy for every command below.

Execute the following commands in PowerShell 7:

```
Set-Location -LiteralPath $workingRoot
```

Your PowerShell prompt must show the working-copy path.

Execute the following commands in PowerShell 7:

```
pwsh -NoProfile -Command `
"New-Item -ItemType Directory -Force -Path 'records\sessions\2026-02-
16\baseline\ID2-20260216-0900-test-run\artifacts' | Out-Null"
```

Layout rule (important):

- `session.json` lives in the **session root**
- module JSON records live in the **artifacts** folder

This structure is created automatically by the following commands.

Step B: Copy templates into the baseline scaffold (robocopy)

This copies **only** the two templates from:
`records\templates`

…and places them in the correct baseline locations, then renames them to the filenames used in the workflow.

B1) Session template → session.json (session root)

Execute the following commands in PowerShell 7:

```
pwsh -NoProfile -Command `
"robocopy 'records\templates' 'records\sessions\2026-02-16\baseline\ID2-
20260216-0900-test-run' 'session.template.json' /NFL /NDL /NJH /NJS /NC
/NS /NP /R:1 /W:1 | Out-Null;

 if (-not (Test-Path 'records\sessions\2026-02-16\baseline\ID2-20260216-0900-
test-run\session.json')) { `
   Rename-Item -Path 'records\sessions\2026-02-16\baseline\ID2-20260216-0900-
test-run\session.template.json' -NewName 'session.json' -Force `
 }"
```

B2) Module 03 template → module_03_sapp_simulation_stress_testing_ID_2.json (artifacts)

Execute the following commands in PowerShell 7:

```
pwsh -NoProfile -Command `
"robocopy 'records\templates' 'records\sessions\2026-02-16\baseline\ID2-
20260216-0900-test-run\artifacts' 'module_03.template.json' /NFL /NDL
/NJH /NJS /NC /NS /NP /R:1 /W:1 | Out-Null;

 if (-not (Test-Path 'records\sessions\2026-02-16\baseline\ID2-20260216-0900-
test-run\artifacts\module_03_sapp_simulation_stress_testing_ID_2.json')) { `
   Rename-Item -Path 'records\sessions\2026-02-16\baseline\ID2-20260216-0900-
test-run\artifacts\module_03.template.json' -NewName
'module_03_sapp_simulation_stress_testing_ID_2.json' -Force `
 }"
```

Step C: Create a matching variant scaffold - time pressure. (Recommended for test run)

Target variant artifacts folder:

```
$workingRoot\records\sessions\2026-02-16\variants\ID2-20260216-
0900-test-run-timepressure\artifacts
```

C1) Create variant folders

Execute the following commands in PowerShell 7:

```
pwsh -NoProfile -Command `
"New-Item -ItemType Directory -Force -Path 'records\sessions\2026-02-
16\variants\ID2-20260216-0900-test-run-timepressure\artifacts' | Out-Null"
```

C2) Copy templates into the variant scaffold (robocopy + rename)

Execute the following commands in PowerShell 7:

```
pwsh -NoProfile -Command `
"robocopy 'records\templates' 'records\sessions\2026-02-16\variants\ID2-
20260216-0900-test-run-timepressure' 'session.template.json' /NFL /NDL
/NJH /NJS /NC /NS /NP /R:1 /W:1 | Out-Null;

 if (-not (Test-Path 'records\sessions\2026-02-16\variants\ID2-20260216-0900-
test-run-timepressure\session.json')) { `
   Rename-Item -Path 'records\sessions\2026-02-16\variants\ID2-20260216-0900-
test-run-timepressure\session.template.json' -NewName 'session.json' -Force `
 }"

pwsh -NoProfile -Command `
"robocopy 'records\templates' 'records\sessions\2026-02-16\variants\ID2-
20260216-0900-test-run-timepressure\artifacts' 'module_03.template.json'
/NFL /NDL /NJH /NJS /NC /NS /NP /R:1 /W:1 | Out-Null;

 if (-not (Test-Path 'records\sessions\2026-02-16\variants\ID2-20260216-0900-
test-run-
timepressure\artifacts\module_03_sapp_simulation_stress_testing_ID_2.json'))
{ `
   Rename-Item -Path 'records\sessions\2026-02-16\variants\ID2-20260216-0900-
test-run-timepressure\artifacts\module_03.template.json' -NewName
'module_03_sapp_simulation_stress_testing_ID_2.json' -Force `
 }"
```

Quick verification (recommended)

Confirm the two key files exist where the scripts expect them:

Baseline folder check

Execute the following commands in PowerShell 7:

```
Get-ChildItem "records\sessions\2026-02-16\baseline\ID2-20260216-0900-test-
run" -Force
Get-ChildItem "records\sessions\2026-02-16\baseline\ID2-20260216-0900-test-
run\artifacts" -Force
```

Variants folder check

Execute the following commands in PowerShell 7:

```
Get-ChildItem "records\sessions\2026-02-16\variants\ID2-20260216-0900-test-
run-timepressure" -Force
Get-ChildItem "records\sessions\2026-02-16\variants\ID2-20260216-0900-test-
run-timepressure\artifacts" -Force
```

Transition note: now scripts operate over a stable session

At this point you have a deterministic session container:

- `session.json` anchors the run metadata and session identity
- `module_03_sapp_simulation_stress_testing_ID_2.json` anchors the structured stress-testing record
- the directory layout supports normalization, export summaries, and later comparative analysis without path ambiguity

Proceed to the script-driven workflow sections.

Important Practice During Session Runs (ID_2 Operational Hygiene)

1. If Any Structural Errors Occur — Run Normalization

If any persistence guard or schema warnings occur during Module 03 population or comparison, run the normalization script **after auto-save and before proceeding to the next step**.

Normalization enforces canonical JSON shape (arrays for simulation_runs and all list-typed fields). This step is structural only. It does not change governance reasoning content, conclusions, or posture. Normalization corrects schema shape drift, not governance substance.

If an error occurs before save, discard the partial file and restart the session from a clean scaffold.

---

2. Semicolon Entry Rule (When Using Script Prompts)

When entering multiple items in script prompts:

- Separate items using a semicolon followed by a single space (;).
- Do not place a semicolon at the end of the line.
- Each item should be a complete phrase.

Correct example:

```
Time pressure; Conflicting telemetry; Stakeholder request for clarification
```

Incorrect examples:

```
Time pressure;Conflicting telemetry;
Time pressure; Conflicting telemetry;
```

Trailing semicolons can produce empty array elements.

---

3. Sentence Completion Rule

Where prompts expect narrative text (e.g., scenario summary, context notes, objective):

- Write complete sentences.
- End punctuation is optional, but consistency is recommended.
- Avoid fragment-only entries unless the field explicitly expects short list items.

This supports:

- Consistent JSON serialization
- Clear narrative capture
- Professional formatting in exported summaries

How Semicolon Entries Serialize to JSON (important)

When entering multiple items in a prompt using semicolon separators:

Good example of separation, semicolon separators and no full stop at the end.

```
Misinterpretation of percentage as quality or safety indicator; Pressure-driven publication of incomplete metrics; Erosion of uncertainty transparency in governance reporting; Scope creep into recommendation-making without mandate; Incorrect escalation decisions due to ambiguous thresholds
```

This will serialize in the JSON file as:

```
"potential_harms": [
  "Misinterpretation of percentage as quality or safety indicator",
  "Pressure-driven publication of incomplete metrics",
  "Erosion of uncertainty transparency in governance reporting",
  "Scope creep into recommendation-making without mandate",
  "Incorrect escalation decisions due to ambiguous thresholds"
]
```

Formatting rules applied:

- Each semicolon-separated item becomes a distinct array element.
- A semicolon followed by a single space (;) ensures clean splitting.
- No trailing semicolon is used.

- Each entry is a complete phrase or sentence.

If any structural errors occur during a session restart, or after an auto-save error, run the normalization script before proceeding.

It ensures:

- `simulation_runs` is stored as an array ([{…}]), even if only one run exists.
- All list-typed fields are arrays of strings.
- Scalar list drift is corrected.
- Object-to-array mismatches are resolved.

Normalization is structural only.
It does not alter captured governance reasoning content or ID_2 posture.

STEP 1 — Populate the Baseline Session Record

This step creates the container for your reasoning run and establishes governance posture. It does not perform stress testing. The session record must remain:

- `non_inspection_session_record`
- `access_mode_primary = ID_2`
- `inspection_status = not_frozen`
- `human_decision_required = true`

These fields preserve ID_2 posture.

Copy/paste/enter to start session – ***TIP write down (record) all input for baseline session, used for variant session input and to vary input slightly for comparison section outputs.***

Execute the following commands in PowerShell 7:

```
pwsh .\scripts\helpers\Edit-ID2SessionJson.ps1 `
  -Path "records\sessions\2026-02-16\baseline\ID2-20260216-0900-test-run\session.json"
```

---

1. Session ID

Prompt:

```
Session ID (recommend: ID2-YYYYMMDD-HHMM-<slug>)
```

What to enter:
A unique identifier using this structure for all future runs:

ID2-YYYYMMDD-HHMM-<short-description>

For test run we use this:

```
ID2-20260216-0900-test-run
```

As this toolkit is pre-configured.

Purpose:

Ensures reproducibility and anchors baseline/variant comparisons.

This ID should never imply approval, adequacy, or compliance.

---

2. Timezone

Prompt:

```
Timezone
```

What to enter:
The timezone in which the session is conducted.

Example format:

```
Australia/Brisbane
```

Purpose:
Ensures timestamps are interpreted correctly across runs.

---

3. Facilitator Name

Prompt:

```
Facilitator name
```

What to enter:
The individual responsible for guiding the session.

Purpose:
Establishes explicit human accountability, required in ID_2.

---

4. Facilitator Role

Prompt:

```
Facilitator role
```

What to enter:
The governance or operational role the facilitator holds.

Example:

```
AI Governance Lead
Workshop Facilitator
Operational Reviewer
```

Purpose:
Clarifies authority context (not enforcement authority).

---

5. Participant Name(s)

Prompt:

```
Participant #1 name
Participant #1 role
```

What to enter:
Individuals participating in the reasoning process.

If solo:

- Participant can match facilitator.
- Role can remain blank.

Purpose:
Captures who contributed to reasoning signals.

---

6. Project Name

Prompt:

```
Project name
```

What to enter:
A short neutral label describing the subject of the run.

Example:

```
AI Usage Metric Review
Internal Governance Reporting
```

Avoid:

- “Compliance validation”
- “Certification review”
- “Audit conclusion”

ID_2 is exploratory.

---

7. Audience

Prompt:

```
Audience (e.g., internal; workshop; standards)
Enter items separated by ';'
```

What to enter:
The intended audience for the session outputs.

Use semicolon-separated values.

Example:

```
internal; workshop
```

Stored as an array. Defines communication context, not regulatory scope.

---

8. Purpose Statement

Prompt:

```
Purpose statement
```

Default:

```
Shift governance from documentation to operational behavior (ID_2).
```

You may refine, but preserve operational reasoning, non-inspection posture, and signals-only framing.

Purpose:
Defines why the session exists — not what it concludes.

---

9. In Scope

Prompt:

```
In scope (use ';' between items)
```

What to enter:
Operational elements that will be reasoned about.

Examples:

- AI-assisted drafting; tools; etc..
- Telemetry review;
- Stakeholder communication under pressure;

Do not include:

- Legal conclusions
- Compliance certification
- Regulatory enforcement

Purpose:
Defines reasoning boundary.

---

10. Out of Scope

Default includes:

- Legal advice;
- Compliance determinations;
- Certification/approval language;
- Automated enforcement decisions;

These must remain to preserve ID_2 posture.

---

11. Run Plan Notes

Optional.

Use to document:

- Why this run exists;
- What stress dimension will be tested later;

This is informational only.

---

12. Artifacts List

Leave blank for baseline.

Artifacts are generated later.

---

13. Signed By (Optional)

This is not mandatory at baseline creation.

It becomes relevant when:

- The session is complete
- You want to lock accountability

ID_2 does not require inspection freeze.

What Happens When You Save

The script writes session.json, creates a timestamped backup, and preserves non-inspection posture.

The result should structurally resemble the template session structure session.

Baseline Session Is Now Established

At this stage you have:

`Baseline\ID2-20260216-0900-test-run\session.json` – file.

No stress simulation has occurred yet. The next documented step is:

Create and populate Module 03 baseline record (stress simulation).

STEP 2 — Populate Module 03 (Baseline Simulation)

This is where operational reasoning begins.

Module 03 captures how governance reasoning behaves under defined stress conditions. It does not assert adequacy, compliance, or approval.

Execute the following commands in PowerShell 7:

```
pwsh .\scripts\helpers\Map-VersionD-StressTo-ID2Module03.ps1 `
  -Path "records\sessions\2026-02-16\baseline\ID2-20260216-0900-test-
run\artifacts\module_03_sapp_simulation_stress_testing_ID_2.json"
```

You are now populating a non_inspection_module_record for `module_03_sapp_simulation_stress_testing_ID_2.json`.

The following sections explain what each field captures and how it should be completed.

SECTION A — Inputs

These define the starting conditions of the stress simulation.

---

1. Scenario Seeds

Purpose:
Describe the operational situation being reasoned about.

These are neutral descriptions, not judgments.

What to enter:
Short factual statements describing:

- The AI-related context
- The governance setting
- The reporting or operational condition

Examples of structure (not content):

- Organisation reports X metric;
- Governance requires monitoring of Y;
- Stakeholder requests interpretation of Z;

Avoid:

- “This is compliant.”
- “This is acceptable.”
- “This meets requirements.”

These are not allowed in ID_2.

---

2. Stressors

Purpose:
Define what pressures or distortions are being introduced.

Examples of stress dimensions (structure only):

- Time pressure;
- Conflicting telemetry;
- Stakeholder demand;
- Ambiguous definitions;
- Delegation ambiguity;

Stressors simulate pressure on reasoning, not system failure.

---

3. Context Notes

Purpose:
Explain why this simulation is being run.

This is narrative context.
It should clarify:

- What kind of governance behavior is being explored
- That the purpose is exploratory
- That no adequacy determination is being made

Example:

```
Baseline stress test of AI usage percentage reporting under
operational pressure to identify fragility in scope boundaries, signal
interpretation, and escalation triggers without making adequacy or
compliance determinations.
```

SECTION B — Captures

This defines the reasoning boundary and guardrails.

---

4. Objective

Purpose:
State what the simulation seeks to observe.

Examples of structure:

- Determine how reasoning shifts under time pressure.
- Observe how signal interpretation degrades under ambiguity.
- Identify fragility in scope boundaries.

This is observational — not evaluative.

---

5. Scope In

Purpose:
Define what reasoning elements are included.

Examples of structure:

- AI-assisted drafting;
- Telemetry review;
- Internal reporting logic;
- Stakeholder communication framing;

---

6. Scope Out

Must exclude:

- Legal advice;
- Compliance determination;
- Certification/approval;

- Enforcement decisions;

These preserve ID_2 posture.

---

7. Assumptions

Purpose:
Make hidden conditions explicit.

Examples of structure:

- Telemetry may be incomplete;
- Stakeholders may interpret metrics differently;
- Definitions remain stable during the run;

Assumptions reduce silent reasoning drift.

---

8. Evidence to Capture

Purpose:
Specify what signals you will observe during stress.

Examples of structure:

- Wording shifts;
- Removal of uncertainty;
- Threshold language appearing;
- Escalation discussions;
- Scope expansion attempts;

Defines what qualifies as signal drift.

---

9. Controls & Guardrails

These should always include:

- No adequacy language;
- No sufficiency language;

- No compliance claims;
- No approval framing;
- Human review required before external use;

These preserve the ID_2 non-inspection constraint.

SECTION C — Simulation Run

This is the actual operational reasoning trace.

The script creates simulation_runs.

Important: this must be an **array**, even if there is only one run.

---

10. Run ID

Typically:

RUN-01

If you create variants later:

RUN-02
RUN-03

---

11. Scenario Summary

Purpose

Describe what happened during the simulation under the defined stress conditions.

This is a neutral narrative.
It explains context, not outcomes.

What it should contain

- The operational context
- The stress environment
- The type of reasoning activity
- Whether escalation occurred (without judgment)

Example (Baseline Form)

```
Baseline simulation of AI-assisted content reporting under defined
scope boundaries. Telemetry inputs were reviewed while responding to
stakeholder clarification requests. The session focused on
interpreting the metric and maintaining uncertainty discipline without
triggering formal escalation.
```

What to avoid

- “This was acceptable.”
- “This met governance standards.”
- “This failed review.”

---

12. Stress Conditions — Example Sets

A. Time Pressure

Use when reasoning occurs under urgency.

- Compressed review timeline;
- Same-day publication expectation;
- Reduced peer review window;
- Deadline-driven stakeholder escalation;
- Limited opportunity to reconcile telemetry before response;

Purpose:
To observe how urgency affects wording discipline and uncertainty articulation.

B. Telemetry Ambiguity

Use when input signals are inconsistent or incomplete.

- Conflicting telemetry classifications;
- Inconsistent definitions of “AI-assisted”;
- Partial or delayed telemetry reconciliation;
- Unstable metric calculation source;
- Metric derived from non-harmonized data sources;

Purpose:
To observe how ambiguity influences signal interpretation.

C. Stakeholder Pressure

Use when interpretation expectations influence reasoning.

- Stakeholder request for immediate clarity;
- Demand for recommendation framing;
- Expectation to interpret metric as risk indicator;
- Implicit request for adequacy confirmation;
- Leadership demand for simplified explanation;

Purpose:
To observe drift toward endorsement or threshold language.

D. Definition Drift

Use when terminology stability is under strain.

- Unclear definition of metric boundaries;
- Ambiguity between reporting and evaluation;
- Conflation of operational metric with governance maturity;
- Expanding scope of metric interpretation;
- Informal redefinition during discussion;

Purpose:
To test scope boundaries.

E. Authority Boundary Stress

Use when role clarity is challenged.

- Unclear escalation threshold;
- Ambiguous decision ownership;
- Delegated interpretation without formal mandate;
- Informal consensus replacing documented escalation;
- Role confusion between reviewer and approver;

Purpose:
To observe authority drift.

F. Communication Friction

Use when drafting or framing pressures exist.

- Pressure to remove uncertainty qualifiers;
- Preference for simplified executive summary language;
- Removal of nuance for readability;
- Reduction of caveats to improve clarity;
- Tone shift toward reassurance;

Purpose:
To detect language-based drift.

How to Select Stress Conditions

In a baseline simulation:

- Choose 2–4 stress conditions
- Keep them moderate
- Ensure they align with declared stressors

In a variant:

- Intensify one dimension
- Add 1–2 new conditions
- Do not remove baseline conditions unless intentionally testing replacement

---

Example Baseline Stress Conditions (Clean ID_2 Form)

- Compressed review timeline
- Stakeholder request for clarification of metric meaning
- Conflicting telemetry classifications

These align structurally with the canonical example

`module_03_sapp_simulation_stress_testing_ID_2.json`

Important Guardrail Reminder

Stress conditions should:

- Describe pressure
- Not describe outcome
- Not imply failure
- Not imply adequacy
- Not imply compliance

Correct:

“Compressed review timeline”

Incorrect:

“Inadequate review process”

---

13. Branch Points

Purpose

Identify moments where reasoning could have gone in multiple directions.

Branch points reveal structural fragility.

Example

- Whether to include a threshold recommendation
- Whether to delay reporting due to telemetry ambiguity
- Whether to escalate to governance lead
- Whether to reframe the metric as a risk indicator

These are not decisions made — they are decision junctures observed.

---

14. Signals Observed

Purpose

Document observable reasoning artifacts under stress.

These are evidence-based signals.

Example

- Draft language shifted toward recommendation phrasing
- Uncertainty qualifiers shortened in summary text
- Metric referenced as governance maturity proxy
- Stakeholder urgency increased frequency of clarification

Signals describe reasoning behavior; they do not indicate correctness or adequacy.

---

15. Fragility Points

Purpose

Identify where reasoning stability weakened.

Fragility is not failure. It marks sensitivity to pressure.

Example

- Metric interpreted as qualitative indicator without explicit definition
- Scope boundary between reporting and evaluation blurred
- Telemetry inconsistency reduced interpretive confidence
- Escalation thresholds were not clearly documented

Fragility highlights where governance may degrade under increased stress.

---

16. Authority Drift Flags

Purpose

Detect subtle shifts toward endorsement, adequacy, or approval language.

This section is central to ID_2.

Example

- Use of phrasing implying acceptable threshold
- Framing metric as evidence of governance adequacy

- Removal of uncertainty qualifiers
- Implicit suggestion that escalation was unnecessary

These flags do not accuse; they mark potential drift toward inspection posture.

---

17. Uncertainty Notes

Purpose

Explicitly preserve ambiguity.

Uncertainty must be preserved.

Example

- Definition of “AI-assisted” may vary across telemetry sources
- Metric does not indicate quality or safety level
- Stakeholder interpretation may differ from governance intent
- Time pressure influenced drafting tone

If uncertainty disappears under stress, that itself becomes a signal.

---

18. Escalation Triggers

Purpose

Define what would require human escalation.

These are not automatic actions.
They are thresholds for human intervention.

Example

- Introduction of recommendation language without mandate
- Persistent telemetry inconsistency
- Stakeholder demand for adequacy framing
- Removal of uncertainty qualifiers from public draft

Triggers indicate when governance boundaries may require human review.

---

19. Stop or Reset Events

Why Stop/Reset Matters

Under stress, reasoning naturally drifts toward simplification, endorsement, or threshold framing.

ID_2 requires documenting whether:

- Drift was noticed,
- Guardrails were re-asserted,
- Scope was clarified before proceeding.

A simulation that documents no stop events should still confirm that posture was actively monitored.

A simulation that documents stop/reset events demonstrates governance reflex discipline.

What Not to Do

Do not write:

- “Failure occurred.”
- “Process broke down.”
- “Governance collapsed.”

Instead write:

- “Drift was identified and guardrails were re-asserted.”
- “Language was corrected.”
- “Scope was re-declared.”

ID_2 documents reflexes, not verdicts.

Two Types of Events

No Stop / No Reset

This is common in baseline runs.

It means:

- Stress occurred
- Drift signals appeared
- But no guardrail was breached
- Reasoning remained within declared scope

Example (Expanded)

- No stop event triggered.
- All observed authority drift signals were documented without converting into recommendation language.
- Posture remained aligned with declared guardrails.

This shows monitoring occurred — not complacency.

Stop Event (Pause)

A stop event occurs when:

- Recommendation language appears
- Adequacy framing is introduced
- Scope expands beyond declared boundary
- Escalation authority becomes ambiguous

A stop event does not mean failure.
It means the guardrail worked.

Example (Expanded)

- Simulation paused when draft language introduced an implied acceptable threshold.
- The session re-affirmed that ID_2 does not permit adequacy framing.
- Language was revised before continuing.

This shows boundary enforcement in action.

Reset Event (Scope Re-declared)

A reset is stronger than a pause.

It means:

- The simulation conditions changed materially
- Scope required clarification
- Escalation path required redefinition

Example (Expanded)

- Scope reset triggered when stakeholder request expanded into compliance interpretation.
- Session re-declared scope_out parameters.
- Escalation triggers were re-confirmed before proceeding.

This shows governance re-anchoring itself.

---

20. Ethical Constructs

Optional but recommended.

Purpose

Deepen interpretability by identifying tensions and tradeoffs.

A. Tensions

- Speed vs review depth
- Clarity vs nuance
- Transparency vs reassurance
- Metric simplicity vs interpretive complexity

B. Affected Parties

- Governance team
- Business stakeholders
- Content authors
- End users
- Risk and compliance functions

C. Potential Harms

- Misinterpretation of metric as safety indicator
- Publication of incomplete data
- Erosion of uncertainty transparency
- Scope creep into recommendation authority

D. Tradeoffs Noted

- Speed vs telemetry reconciliation
- Stakeholder reassurance vs internal caution
- Simplicity vs accuracy

Ethical constructs expand understanding — they do not assign blame.

After completing and auto save is confirmed (next create a variant session).

Saved: ...\session.json

Backup: ...\session.json.bak.<timestamp>

*If structural warnings appear, refer to **Troubleshooting section at the end**— Schema Warning before continuing.*

Important ID_2 Reminder

At the end of Module 03 baseline session:

- completion.status remains "draft"
- No adequacy claims are made
- No compliance claims are made
- No certification language appears
- Human review remains required

This preserves the ID_2 non-inspection boundary

`module_03_sapp_simulation_stress_testing_ID_2.json`

Next step:

STEP 3 — Create and Populate Variant (Time-Pressure Intensified)

The variant repeats the baseline process but introduces intensified stress for comparison.

The objective is not a different conclusion.
The objective is to test:

- Boundary stability
- Authority drift under pressure
- Escalation reflexes
- Uncertainty preservation

ID_2 posture remains unchanged:

- non_inspection_session_record
- access_mode_primary = ID_2
- inspection_status = not_frozen
- human_decision_required = true

Variant File Discipline

The variant Module 03 record must be created from the template.

Do not copy the populated baseline Module 03 into the variant folder.

Each variant must be independently populated using the mapper prompts to preserve:

- Clean stress isolation,
- Accurate comparative analysis,
- Proper governance reasoning discipline,
- Structural integrity of JSON records.

Variants intensify stress conditions — they do not inherit reasoning artifacts.

3.1 Create the Variant Session Folder

Create a new folder under `variants` using the baseline Session ID plus a stress descriptor.

If created (on page 3 to 6 robocopy) **and verified** (on page 6) **skip 3.1 - 3.2 and run 3.3.**

`records\sessions\`**`2026-02-16`**`\variants\`**`ID2-20260216-0900-test-run-timepressure`**`\`

3.2 Session Template Copy — Future Use Guidance

For future runs (beyond this test example), you may use either:

Option A — Copy-Item (Simpler)

Recommended when creating a single new session:

```
Copy-Item `
  "records\templates\session.template.json" `
  "records\sessions\<DATE>\variants\<NEW_SESSION_ID>\session.json"  `
  -Force
```

Replace `<DATE>` and `<NEW_SESSION_ID>` with actual values.

This method is concise and suitable for manual session setup.

Option B — Robocopy (Template Fidelity)

Recommended when:

- Maintaining consistent release structure
- Working in batch environments
- Avoiding partial copies
- Preserving naming discipline across multiple runs

```
robocopy "records\templates" `
         "records\sessions\<DATE>\variants\<NEW_SESSION_ID>" `
         "session.template.json" `
         /NFL /NDL /NJH /NJS /NC /NS /NP /R:1 /W:1
```

Then rename to `session.json`.

Important

Both methods:

- Copy from canonical templates
- Preserve structural integrity
- Avoid baseline carryover
- Maintain ID_2 posture discipline

For test-run documentation consistency, use the pre-configured command shown below.

3.3 Populate the Variant Session Record – Test-run

Execute the following commands in PowerShell 7:

```
pwsh .\scripts\helpers\Edit-ID2SessionJson.ps1 `
  -Path "records\sessions\2026-02-16\variants\ID2-20260216-0900-test-run-
timepressure\session.json"
```

You will see:

```
=== Edit ID_2 session.json (non-inspection) ===
Tip: press Enter to keep the current value.
```

Field-by-Field Guidance for Variant Session

(Template-Based Test-Run Configuration)

This variant session is created from a blank template. No baseline values are pre-populated.

To preserve comparability:

- Re-enter all baseline-aligned fields manually.
- Do not rely on pressing Enter to inherit values.
- Modify only the intended stress dimension (e.g., time pressure).

For future reuse (editing an existing populated variant file), pressing Enter will preserve prior values.

For learning purposes its recommended to enter your prefilled inputs. For session examples and future runs:

Refer to the file - ***prompt_examples_module_03_sapp_simulation_stress_testing_ID_2.JSON*** *in* `\records\helpers\` *folder.*

The baseline examples are from lines 1-202 and,

the variant examples are from lines 203-409.

Session ID

Enter:

```
ID2-20260216-0900-test-run-timepressure
```

Must match the folder name exactly.

Timezone

Re-enter the same timezone used in the baseline session.

In template-based runs, pressing Enter will leave the field empty.

Facilitator Name / Role

Re-enter the same facilitator name and role used in the baseline session unless intentionally changed.

The variant session remains independently accountable.

Participant Name(s)

Re-enter participant details from the baseline unless intentionally changed.

Comparability requires identical participant context unless testing participation changes.

Project Name

Re-enter the same project name as baseline.

The project context must remain stable for clean comparison.

Audience

Re-enter the same audience values used in the baseline session.

Audience context should remain constant between baseline and variant.

Purpose Statement (Variant)

The variant purpose statement must explicitly identify the intensified stress dimension.

Enter:

```
Variant run applying intensified time-pressure stressor for comparison
against baseline.
```

The purpose statement should:

- Identify the specific stress dimension being intensified
- Preserve exploratory, non-inspection posture
- Avoid introducing new scope elements
- Avoid compliance, adequacy, or approval framing

The variant purpose defines stress differentiation only. It does not redefine scope, posture, or governance authority.

In Scope

Re-enter the same scope boundaries as baseline.

Variant stress must not alter scope.

Scope stability is required for valid comparison.

Out of Scope

Re-enter the same out-of-scope exclusions as baseline.

Out-of-scope constraints must remain unchanged to preserve ID_2 posture.

Run Plan Notes (Variant)

Run Plan Notes distinguish the variant from the baseline.

They should:

- Identify the intensified stress dimension
- Confirm scope remains unchanged
- Preserve non-inspection posture
- Support later comparative traceability

Example entries:

- Intensified time-pressure scenario for comparison analysis
- Variant run isolating intensified time-pressure stress while preserving scope and guardrails
- Variant run intensifying time pressure to evaluate boundary stability and authority drift against baseline

Run Plan Notes clarify intent — they do not introduce governance conclusions.

Artifacts List

Leave blank.

Artifacts are generated later in the comparison pipeline.

Signed By

Leave blank at this stage.

Signing occurs only after the comparison pipeline is complete.

Save Confirmation

The script will display:

```
Saved:  ...\session.json
Backup: ...\session.json.bak.<timestamp>
```

At this point, the variant session container is complete.
Proceed to Step 3.4 or 3.5.

If a Persistence Guard Warning Appears (Test-Run Only) — Run Normalization

If you see:

```
Persistence guard: simulation_runs is not an array immediately before save.
```

Run normalization for the **2026-02-16 test-run folder only**:

```
pwsh .\scripts\primary\Normalize-ID2Module03.ps1 `
  -RootPath "records\sessions\2026-02-16" `
  -InPlace -Backup
```

Purpose: Enforce canonical JSON shape for Module 03 records under the 2026-02-16 test-run (arrays for `simulation_runs` and list-typed fields).
Note: Normalization is structural only. It does not change governance content, conclusions, or posture.

Optional verification (recommended) after the auto save

Execute the following commands in PowerShell 7:

```
$path = "records\sessions\2026-02-16\variants\ID2-20260216-0900-test-run-
timepressure\artifacts\module_03_sapp_simulation_stress_testing_ID_2.json"
; ((Get-Content -Raw $path | ConvertFrom-Json -Depth 80).simulation_runs -
is [System.Array])
```

Expected result:

```
True
```

3.4 Create the Variant Module 03 Record

The variant Module 03 record must be created from the canonical template.

Do **not** copy the populated baseline Module 03 file into the variant folder.
Each variant must begin from a clean template to preserve stress isolation and accurate comparison.

If created (on page 3 to 6 robocopy) **and verified** (on page 6) **skip 3.4 and run 3.5.**

```
records\sessions\2026-02-16\variants\ID2-20260216-0900-test-run-
timepressure\artifacts\module_03_sapp_simulation_stress_testing_ID_2
```

Test-Run Command (Configured Example)

Execute:

```
Copy-Item
  "records\templates\module_03.template.json" `
  "records\sessions\2026-02-16\variants\ID2-20260216-0900-test-run-
timepressure\artifacts\module_03_sapp_simulation_stress_testing_ID_2.json" `
  -Force
```

Future Use (Custom Session)

For new sessions, update `<DATE>` and `<NEW_SESSION_ID>`:

```
Copy-Item
  "records\templates\module_03.template.json" `

"records\sessions\<DATE>\variants\<NEW_SESSION_ID>\artifacts\module_03_sapp_si
mulation_stress_testing_ID_2.json" `
  -Force
```

Robocopy Alternative (Optional)

For batch runs or strict template-fidelity workflows:

```
robocopy "records\templates"
         "records\sessions\<DATE>\variants\<NEW_SESSION_ID>\artifacts"
         "module_03.template.json" `
         /NFL /NDL /NJH /NJS /NC /NS /NP /R:1 /W:1
```

Then rename:

```
Rename-Item
"records\sessions\<DATE>\variants\<NEW_SESSION_ID>\artifacts\module_03.templat
e.json" `
  -NewName "module_03_sapp_simulation_stress_testing_ID_2.json" -Force
```

Why This Step Matters

Creating the variant from template ensures:

- Clean simulation record
- No baseline reasoning copied unintentionally
- Independent stress capture
- Accurate comparative deltas

Only after this file exists should you proceed to Step 3.5.

At this stage both baseline and variant contain:

- session.json
- artifacts\module_03_sapp_simulation_stress_testing_ID_2.json.

Structure should be identical.

Next step:

3.5 — Populate Variant Module 03 (Intensified Stress)

This step populates the variant Module 03 using the same structure as baseline, but under intensified stress.

Goal:

- Observe boundary stability under pressure
- Capture drift signals (not conclusions)
- Preserve ID_2 posture

Execute the following commands in PowerShell 7:

```
pwsh .\scripts\helpers\Map-VersionD-StressTo-ID2Module03.ps1 `
  -Path "records\sessions\2026-02-16\variants\ID2-20260216-0900-test-run-
timepressure\artifacts\module_03_sapp_simulation_stress_testing_ID_2.json"
```

Guiding Rule for Variants

In a time-pressure variant:

- Stressors intensify

- Signals may appear earlier
- Authority drift flags may increase
- Stop/reset events may occur more readily

What must not change:

- Scope boundaries
- Out-of-scope exclusions
- Guardrails
- Non-inspection posture

SECTION A — Inputs (Variant)

1) Scenario Seeds

What to do:
Re-enter the same scenario seeds used in the baseline.

Because the variant Module 03 record is created from a blank template

`module_03_sapp_simulation_stress_testing_ID_2.json`

, the fields will be empty. You must manually enter the same scenario seeds again to preserve comparability.

Purpose:
Comparative analysis requires a stable scenario. The scenario remains constant; only stress intensity changes.

Important:
“Keep consistent” means identical wording — not pressing Enter.

In variants, comparability is achieved by re-entering identical scenario seeds while modifying only the stress intensity variables.

---

2) Stressors to Inject (Variant)

What to do:
Re-enter the baseline stressors and intensify the time-pressure dimension through wording.

Stressors define what pressures are injected — not outcomes.

The variant should isolate one intensified dimension while keeping scope and posture unchanged.

- Severe time pressure
- Same-day stakeholder response requirement
- Reduced peer review window
- Deferred telemetry reconciliation due to deadline

Purpose: Stressors define *what pressures are injected*, not what happened.

---

3) Context Notes (Variant)

What to do: State why this variant exists and what dimension is intensified.

Example structure:

Variant run to observe governance boundary drift under intensified time pressure, including compressed review windows and reduced opportunity to reconcile telemetry, without making adequacy or compliance determinations.

Keep the full stop at the end of the sentence.

SECTION B — Captures (Variant)

4) Objective

What to do: Keep the objective aligned with baseline but explicitly include "intensified time pressure."

Example structure:

Observe how governance reasoning and uncertainty discipline shift under intensified time pressure and persistent telemetry ambiguity during metric reporting.

---

5) Scope IN

What to do: Keep the same as baseline.

Reason: If scope changes, you are no longer comparing like-for-like.

---

6) Scope OUT

What to do: Keep the same as baseline.

The variant cannot drift into legal/compliance determinations.

---

7) Assumptions

What to do: Mostly keep baseline assumptions; optionally add variant-specific stress constraints.

Example add-on assumptions (optional):

- Peer review window is materially reduced in this variant
- Stakeholder response deadlines may drive wording simplification

---

8) Evidence to Capture

What to do: Keep baseline evidence items plus any additional "time pressure" observations.

Examples of extra evidence items:

- Removal of uncertainty qualifiers earlier than baseline
- Increased use of reassurance tone
- Increased pressure to provide thresholds/recommendations

---

9) Controls/Guardrails

What to do: Keep identical to baseline (recommended).

Guardrails stabilize posture under stress.

SECTION C — Simulation Run (Variant)

10) Run ID

Use a variant run label. Options:

- Keep `RUN-01` (if each file is a separate run record)
- Or use `RUN-02` to reflect sequencing

Either is acceptable, but be consistent in documentation.

---

11) Scenario Summary (Variant)

What to do: Similar to baseline summary, but explicitly mention intensified time pressure.

Example structure:

Time-pressure variant of the baseline scenario. Review timeline was significantly compressed, and stakeholder demand required immediate interpretation of the metric. Telemetry ambiguity persisted and uncertainty articulation was more difficult to maintain under speed constraints.

---

12) Stress Conditions (Variant)

What to do: These should be **more intense** than baseline.

Good time-pressure variant examples:

- Severely compressed review timeline with same-day publication expectation
- Reduced opportunity for peer review before stakeholder response
- Telemetry reconciliation deferred due to time constraints
- Stakeholder demand for immediate interpretation as governance signal

---

13) Branch Points

What to do: Retain baseline branch points and add time-pressure-specific forks.

Examples of additional forks:

- Whether to publish with unresolved telemetry due to deadline
- Whether to remove uncertainty qualifiers to simplify messaging

---

14) Signals Observed

What to do: Record what you observed happening to reasoning and language under pressure.

Examples:

- Draft wording shifted earlier toward recommendation phrasing
- Uncertainty qualifiers removed earlier in drafting cycle
- Metric referenced more often as proxy for governance maturity
- Escalation discussion deferred due to limited time

---

15) Fragility Points

What to do: Add fragility points that emerge due to intensified stress.

Examples:

- Time constraints reduced opportunity to clarify definitions before communication
- Boundary between reporting metric and governance judgment blurred more rapidly
- Peer review limitations reduced detection of drift language

---

16) Authority Drift Flags

What to do: Capture endorsement/approval tone drift that appears under pressure.

Examples:

- Increased tolerance for recommendation-style phrasing under time constraints
- Implicit suggestion that escalation is unnecessary due to deadline pressure
- Removal of uncertainty qualifiers from stakeholder-facing summary
- Draft language implying the metric indicates adequacy

---

17) Uncertainty Notes

What to do: Preserve uncertainty explicitly. Add time-pressure-specific uncertainty.

Examples:

- Time pressure likely increased influence on tone and wording decisions

- Peer review window limitations may reduce drift detection
- Escalation thresholds were not re-evaluated under compressed timeline

---

18) Escalation Triggers

What to do: Keep baseline triggers plus time-pressure-specific triggers.

Add example:

- Deadline-driven pressure to bypass formal review checkpoints

---

19) Stop or Reset Events

What to do: If drift occurred, document whether you paused or reset.

Examples:

- Simulation paused when implied threshold language appeared; wording corrected before continuing
- No formal stop triggered; heightened monitoring required under deadline pressure

This demonstrates governance reflex discipline.

---

20) Ethical Constructs (Variant)

What to do:
Retain the full baseline ethical construct structure.
Add deadline-specific tensions, harms, and trade-offs to reflect intensified time pressure.

The variant must:

- Preserve baseline tensions for comparability.
- Add intensified stress-specific dimensions.
- Remain exploratory (no conclusions).
- Avoid adequacy or compliance framing.

Tensions

Baseline tensions remain.
Add deadline-specific tension, for example:

- Deadline adherence vs procedural rigor.

Potential Harms

Baseline harms remain.
Add intensified deadline-specific harms, for example:

- Publication of incomplete metrics under deadline pressure.
- Reduced opportunity to detect endorsement language under compressed review timelines.

Trade-offs Noted (Variant)

Baseline trade-offs remain in place for comparability.

Add intensified balancing dynamics specific to the time-pressure condition. For example:

- Deadline adherence vs full telemetry reconciliation
- Rapid stakeholder response vs deliberate escalation review

Trade-offs document how competing pressures were balanced under stress.
They do not imply governance failure, adequacy, compliance, or approval conclusions.

Post-Save Structural Check (Before Tagging)

After completing Module 03 prompts and confirming auto-save:

If you see the warning:

```
Persistence guard: simulation_runs is not an array immediately before save.
```

Run normalization **before proceeding to tagging or comparison.**

Normalization enforces canonical JSON structure only.
It does not modify governance reasoning content, stress conditions, signals, or posture.

Run:

```
pwsh .\scripts\primary\Normalize-ID2Module03.ps1 `
  -RootPath "records\sessions\2026-02-16" `
  -InPlace -Backup
```

Proceed once canonical structure is confirmed.

Tagging (Immediately After Mapper Save)

If no structural warning appears, run stress-category tagging.

Execute the following commands in PowerShell 7:

```
pwsh .\scripts\helpers\Add-ID2StressCategoryTags.ps1 `
  -Path "records\sessions\2026-02-16\variants\ID2-20260216-0900-test-run-
timepressure\artifacts\module_03_sapp_simulation_stress_testing_ID_2.json" `
  -Tags "authority,telemetry,incentive,ethical,delegation"
```

Tagging should occur **before comparison** to ensure category metadata is present in the canonical structure.

Tagging is informational and does not alter posture.

Variant Completion Checkpoint

At the end of Step 3.5, confirm:

- Variant `module_03_sapp_simulation_stress_testing_ID_2.json` is saved
- `simulation_runs` contains exactly one populated run (array form)
- `"completion": { "status": "draft" }` remains unchanged
- No adequacy, compliance, certification, or approval language appears

If these conditions are satisfied, the variant record is structurally ready for comparison.

Next step

STEP 4 — Comparison Pipeline

(Normalize → Compare → Export Summary)

This step generates an exploratory comparison between baseline and variant runs.
It does **not** assert adequacy, compliance, certification, or readiness.

Operator Sequence (Release Standard)

Perform the following steps in order:

1. Populate baseline Module 03
2. Populate variant Module 03
3. Run normalization (Normalize-ID2Module03.ps1)
4. Run comparative analysis tool
5. Confirm no schema warnings
6. Confirm export JSON generated

Comparison outputs remain exploratory.
No adequacy or compliance claims are permitted.

4.1 Create the Comparison Output Folder

Execute in PowerShell 7:

```
New-Item -ItemType Directory `
  -Path "records\comparisons\2026-02-16\ID2-20260216-0900-test-run" `
  -Force | Out-Null
```

Purpose:
Separates derived comparison artifacts from session records and anchors output to the baseline run.

4.2 Normalize Module 03 JSON Records

Normalization enforces canonical JSON structure before comparison.
It corrects structural drift only (array/object mismatches, scalar list fields).
It does **not** modify reasoning content or posture.

Execute:

```
pwsh .\scripts\primary\Normalize-ID2Module03.ps1 `
  -RootPath "records\sessions\2026-02-16" `
  -InPlace -Backup
```

Expected outcome:

- simulation_runs stored as an array
- All list-typed fields stored as arrays
- .bak backup created

If normalization is skipped, you may see:

```
WARNING: Schema warning: coerced simulation_runs to array
```

If this appears, run normalization before proceeding.

4.3 Run Comparative Analysis and Export Summary

Execute:

```
pwsh .\scripts\primary\ID2_Comparative_Analysis_Tool.ps1 `
  -BaselinePath "records\sessions\2026-02-16\baseline\ID2-20260216-0900-test-
run\artifacts\module_03_sapp_simulation_stress_testing_ID_2.json" `
  -VariantsRoot "records\sessions\2026-02-16\variants" `
  -OutJson "records\comparisons\2026-02-16\ID2-20260216-0900-test-
run\id2_comparison_summary.json"
```

This tool:

- Compares baseline vs variant runs
- Surfaces deltas in:
  - stress_conditions
  - fragility_points
  - authority_drift_flags
  - escalation_triggers
- Generates structured JSON summary
- Includes exploratory-only notes

4.4 Final Verification

Confirm:

- No schema warnings in console
- JSON summary file generated
- Both baseline and variant remain `"completion": { "status": "draft" }`
- No adequacy, compliance, certification, or approval language appears

If these conditions are satisfied, the comparison structure is canonical and release-ready.

- Comparison summary JSON under:

```
records\comparisons\2026-02-16\ID2-20260216-0900-test-
run\id2_comparison_summary.json
```

4.5 Canonical exemplar comparison (full session — recommended for learning)

Use this after completing the **2026-02-16 test run** comparison. This canonical exemplar session **2026-02-10** is included as a reference run.

Canonical exemplar *baseline* root:
`records\sessions\2026-02-10\baseline\ID2-20260210-1835-session\artifacts\module_03_sapp_simulation_stress_testing_ID_2.json`

Canonical exemplar *variants* root:
`records\sessions\2026-02-10\variants`

4.5.1 The exemplar comparison output folder (Done)

Folder and json file is here: `records\comparisons\2026-02-10\ID2-20260210-1835-session\id2_comparison_summary.json`

4.5.2 Normalize the exemplar session day (Done – run only after 4.5.3 step if needed)

```
pwsh .\scripts\primary\Normalize-ID2Module03.ps1 `
  -RootPath "records\sessions\2026-02-10" `
  -InPlace -Backup
```

4.5.3 Run comparative analysis (canonical exemplar)

Run this after right after the test run for full session example:

Execute the following commands in PowerShell 7:

```
pwsh .\scripts\primary\ID2_Comparative_Analysis_Tool.ps1 `
  -BaselinePath "records\sessions\2026-02-10\baseline\ID2-20260210-1835-
session\artifacts\module_03_sapp_simulation_stress_testing_ID_2.json" `
  -VariantsRoot "records\sessions\2026-02-10\variants" `
  -OutJson "records\comparisons\2026-02-10\ID2-20260210-1835-
session\id2_comparison_summary.json"
```

Operator note: This exemplar run is for orientation and repeatability. It remains **exploratory**, **non-inspection**, and **signals-only** like the test-run primer.

Troubleshooting — Schema Warning:

“coerced simulation_runs to array”

Symptom

During comparison you may see:

```
WARNING: Schema warning: coerced simulation_runs to array
```

Meaning

One or more Module 03 files on disk are not stored in canonical array form.

Canonical structure requires:

```
"simulation_runs": [
  { ... }
]
```

Instead, the file was saved as:

```
"simulation_runs": { ... }
```

The comparison tool temporarily corrects this in memory so execution continues. However, the file on disk remains structurally incorrect.

This is a schema issue, not a governance issue.

Required Action

Before running comparison, always normalize:

```
pwsh .\scripts\primary\Normalize-ID2Module03.ps1 `
  -RootPath "records\sessions\2026-02-16" `
  -InPlace -Backup
```

Normalization:

- Enforces canonical array shape
- Corrects scalar-to-array drift
- Preserves reasoning content
- Does not modify posture

Verification (Optional but Recommended)

```
(Get-Content -Raw "<PATH>\module_03_sapp_simulation_stress_testing_ID_2.json"
|
  ConvertFrom-Json -Depth 60).simulation_runs -is [System.Array]
```

Expected result:

```
True
```

If False, re-run normalization.

Important Clarification

This warning does **not** indicate:

- Governance failure
- Invalid reasoning
- Broken comparison
- Posture drift

It indicates schema shape drift only

Canonical JSON Structure — Required Invariants

The following structural rules define a valid Module 03 record.

If violated, treat as schema drift — not governance failure.

Inputs (Arrays of Strings)

The following must always be arrays:

- `inputs.scenario_seeds`
- `inputs.stressors`
- `inputs.references` (may be empty)

Capture Fields (Arrays)

The following must always be arrays:

- `captures.scope_in`
- `captures.scope_out`
- `captures.assumptions`
- `captures.evidence_to_capture`
- `captures.controls_guardrails`
- `captures.method_steps`

`captures.definitions` may remain an object.

Simulation Runs

`simulation_runs` must always be stored as an array:

```
"simulation_runs": [
  { ... }
]
```

Even if only one run exists.

Within each run, the following fields must be arrays:

- `stress_conditions`

- branch_points
- signals_observed
- fragility_points
- authority_drift_flags
- uncertainty_notes
- escalation_triggers
- stop_or_reset_events
- stress_category_tags

Ethical Constructs (If Present)

Each of the following must be arrays:

- tensions
- affected_parties
- potential_harms
- tradeoffs_noted

What Constitutes Schema Drift

- simulation_runs stored as object instead of array
- List-typed fields stored as scalar strings
- Null values where arrays are required
- Nested objects where strings are expected

Schema drift affects comparison stability. It does not alter captured governance reasoning content.

Remediation

Normalization is required before comparison:

```
pwsh .\scripts\primary\Normalize-ID2Module03.ps1 `
  -RootPath "records\sessions\2026-02-16" `
  -InPlace -Backup
```

Mandatory Workflow Order

1. Populate baseline Module 03
2. Populate variant Module 03

3. Normalize Module 03 files
4. Run comparative analysis
5. Export comparison summary

Normalization is mandatory before comparison.

STEP 5 — Summary Checklist (Completion Review)

Use this checklist to confirm the session is structurally sound and comparison-ready.

1. Session Structure

☐ Baseline `session.json` populated
☐ Variant `session.json` populated (Session ID includes variant suffix)
☐ Baseline Module 03 populated
☐ Variant Module 03 populated
☐ Both Module 03 files contain exactly **one** `simulation_runs` entry
☐ Both Module 03 files remain `"completion": { "status": "draft" }`
☐ No adequacy, compliance, certification, or approval language appears

---

2. Canonical JSON Integrity

☐ `simulation_runs` is stored as an array ([{…}])
☐ All run list fields are arrays
☐ `inputs.scenario_seeds` and `inputs.stressors` are arrays of strings
☐ No required list fields contain scalar strings or null values
☐ Stress category tags applied to baseline and variant

If structural drift is detected, run normalization before proceeding.

---

3. Comparison Pipeline

☐ Normalize-ID2Module03.ps1 executed
☐ No schema warnings appear in console output
☐ Baseline path correctly references baseline Module 03
☐ VariantsRoot correctly references variants folder
☐ Comparative summary displays differences in:

- Stress conditions
- Fragility points
- Authority drift flags
- Escalation triggers

☐ "Beginner Summary" section generated
☐ Cross-Variant Recurrence section generated

---

4. Export Confirmation

☐ JSON summary exported to:

`records\comparisons\2026-02-16\ID2-20260216-0900-test-run\`**`id2_comparison_summary.json`**

☐ Export includes:

- baseline
- variants_root
- generated_utc
- results[]
- aggregate
- exploratory-only notes

---

5. Posture Confirmation (ID_2 Boundary)

The session remains:

☐ Exploratory
☐ Non-inspection
☐ Signals-only (no decisions)
☐ Human decision required
☐ No inspection freeze applied

Completion Standard

If canonical structure passes, no schema warnings appear, comparison exports successfully, and ID_2 posture is preserved, the session is structurally complete and ready for exploratory reuse.

Concluding Note

Access-Mode ID_2 is a governance reasoning discipline, not a compliance mechanism.

This toolkit provides optional structure for recording, stress-testing, and comparing operational governance signals. It does not change the posture of ID_2, nor does it elevate exploratory outputs into determinations, approvals, or certifications.

ID_2 operates upstream of inspection and enforcement.
Its purpose is to examine how scope boundaries, authority discipline, escalation logic, and uncertainty behave under stress — before formal review or regulatory framing occurs.

The JSON layer enhances repeatability.
The comparison pipeline surfaces signal differences.
Neither substitutes for human interpretation.

Inspection, if required, must occur under a separately declared mode.
ID_2 remains explicitly exploratory.

Current folder setup

Current path revision 11 October 2026. Run this setup first in PowerShell 7, then continue at Step A. It makes a fresh two-candidate working copy and leaves the release files unchanged.

```
$ErrorActionPreference = 'Stop'
$sourceRoot = 'C:\access-pod-v1.0\v1.0-access-mode\ID_2-Access'
$workingRoot = Join-Path $env:TEMP 'Access-PoD-ID2-two-candidate'
if ($PSVersionTable.PSVersion.Major -lt 7) {
  throw 'Use PowerShell 7.'
}
if (Test-Path -LiteralPath $workingRoot) {
  throw 'Choose an unused workingRoot.'
}
New-Item -ItemType Directory -Path $workingRoot | Out-Null
Get-ChildItem -LiteralPath $sourceRoot -Force |
  Where-Object Name -ne 'four-candidates' |
  Copy-Item -Destination $workingRoot -Recurse
Set-Location -LiteralPath $workingRoot
```

If the working folder already exists, choose a new name in $workingRoot and rerun the complete setup. Run later commands in the same PowerShell session. The displayed $workingRoot paths are targets in this copy, not literal release paths.

The walkthrough creates a February 16 primer. The separate February 10 exemplar already contains the supplied baseline and time-pressure candidate. Keep their explicit day paths when comparing. The four-candidates child is excluded from this copy.

Revision provenance

This current guide is derived from the retained Word backup, whose cover is APOD-ID2-SCRIPT-EXAMPLES-002 dated 18 February 2026. The preserved published PDF is APOD-ID2-SCRIPT-EXAMPLES-001 dated 20 February 2026. Its original file is unchanged. This copy updates folder setup and concrete command syntax; the case content and posture remain as written.